

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

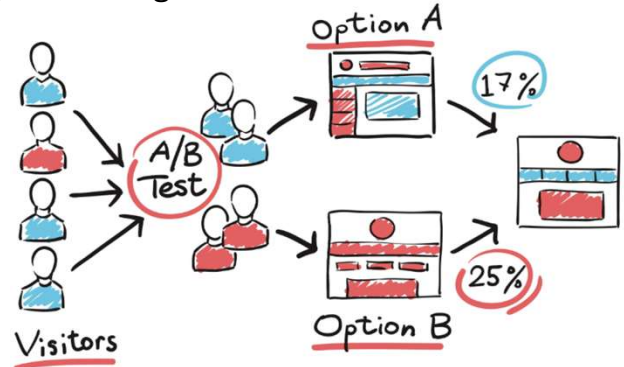
Kyungim Baek

Lecture 9

- Introduction to Probability
- Generative Process

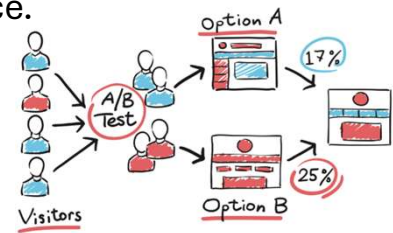
Probability in data science example: A/B testing

- Comparing two versions of a web page to see which one is best for a specific task.
 - E.g., Customer conversion, i.e., converting a customer who browses your site to a paying customer.



Probability in data science example: A/B testing (cont'd.)

- **Randomly** assign users to two groups (A and B).
- Model outcomes (e.g., clicks, conversions, purchases) using **probability distributions**.
- **Estimates parameters** (e.g., conversion rates) from sample data.
- Uses **hypothesis testing** and **confidence intervals** to assess whether observed differences are due to chance.
- Make data-driven **decisions** under uncertainty.



Basic concepts of probability: Experiments

- An experiment is a procedure or *random process* that yields one outcome out of all possible outcomes of the experiment.
- Some examples of experiments are:
 - Tossing a coin and observing the outcome.
 - Rolling a die and observing the outcome.
 - Counting the number of empty slots in a Biki stand.
 - Measuring the weight of a patient in a hospital.

} discrete
→ continuous

Basic concepts of probability: Sample spaces

- The sample space S is the exhaustive set of all possible, disjoint outcomes of an experiment.
- Examples:
 - Heads or tails when tossing a coin $\{H, T\}$
 - When rolling a die, it's the finite set $\{1, 2, 3, 4, 5, 6\}$.
 - The number of Biki slots that can be empty at any time.
 - The range of weights a patient may have.

Basic concepts of probability: Events

- In an experiment, an event E represents a **subset** of the sample space.
- Examples:
 - The coin lands on heads.
 - The die lands 4.
 - The die lands on its odd side (1, 3, or 5).
 - The number of empty Biki slots is 0.
 - The weight of a patient is higher than 140 pounds but less than 200 pounds.

Probability

- Probabilities quantify beliefs.
- Used to measure the uncertainty of an event. For example,
 - How likely is a given person to default on a loan?
 - How unlikely is a given person to donate to a fundraising campaign?
 - How likely is a person to contract the flu this season?
 - How likely is that all the Biki slots at the parking structure will be full?
 - How likely is it that 3 accidents occur on the H1 today?
- We talk of a:
 - null (0) probability when we know an event cannot occur.
 - probability of 1 if an event is certain.

Probability (cont'd.)

- Formally, a probability value is a number that quantifies the likelihood of an event, and which satisfies the following two properties:

$$0 \leq p(E) \leq 1 \quad \text{and} \quad p(S) = 1$$

sample space

- Note that this is not a definition of probability (three axioms).
- A number that is between 0 and 1 is not necessarily a probability unless the sum of all outcomes in the sample space is 1.
- In data analysis, you will most likely assign probabilities to events using probability distributions.
 - A probability distribution is a function that specifies the probability of each outcome (or event) in the sample space.

$\sum_{\omega \in S} p(\omega) = 1$ (discrete)
 $\int_S f(x) dx = 1$ (continuous)

Random samples

- Definition of a random sample in a mathematical and formal context:

- Random sampling from a **population**: Let P represent a population consisting of N elements.

A subset R of P ($R \subseteq P$) is a random sample if each element $x \in P$ has a non-zero probability $p(x)$ of being included in R . This is mathematically represented as: $0 \leq p(x) \leq 1$ for all x in P .

- Example:

P = [all people living in Hawaii]

R = [randomly selected group of Hawaii residents]

$p(x)$ = Probability of any person being selected in the sample

Random samples (cont'd.)

2. Random sampling from a **repeated experiments**: Consider an experiment that is repeatable *under identical conditions*. Let $O = o_1, o_2, \dots, o_m$ be the set of outcomes from m repetitions of the experiment. The set O is a random sample of the experiment's possible outcomes.

- Examples:

- E = Flipping a coin

O = [Set of outcomes from flipping the coin multiple times]

Each outcome o_i is an instance of the coin's flip result (heads or tails).

- E = Counting the number of open slots at a Biki station

O = [Counts of the number of open slots at a Biki station over multiple days]

Each outcome o_i is an instance of number of available slots over multiple days.

Random samples (cont'd.)

- The key characteristics of a random sample are:

- **Randomness**: Each member of the population is chosen randomly, meaning every individual has the well-defined chance of being selected. This is often achieved using random number generators or other methods that ensure unbiased selection.
- **Independence**: The selection of one individual or item in the sample does not influence the selection of another. In other words, each selection is an independent event.

Long-run frequencies of an event

- The concept of long-run frequency is an essential aspect of probability.
 - In a casino game where your chances of winning are 1 in 10, you are not guaranteed to win if you participate ten times.
- When flipping a coin – assuming the coin is unbiased – we would intuitively expect to observe **in the long run** as many heads as tails, or about 50% heads and 50% tails.
 - The probabilities of heads and tails are $p(H) = 0.5$ and $p(T) = 0.5$ respectively.
- We can demonstrate this computationally by generating heads (ones) and tails (zeros) using the Python random module.

Recall: The random module

- Implements a variety helpful functions that use randomness.
 - E.g., simulations, shuffling data, sampling, selecting random element(s) from a collection, etc.

```
>>> print(random.choice([0, 1]))  
0  
>>> print(random.choice([0, 1]))  
1  
>>> print(random.choice([0, 1]))  
0
```

fair coin

```
random_trials = [random.choice([0, 1]) for _ in range(100)]  
sum(random_trials)
```

53

```
random_trials = [random.choice([0, 1]) for _ in range(100)]  
sum(random_trials)
```

57

```
random_trials = [random.choice([0, 1]) for _ in range(100)]  
sum(random_trials)
```

46

```
random_trials = [random.choice([0, 1]) for _ in range(100)]  
sum(random_trials)
```

50

Inspecting the number of accumulated heads

- The progression of the number of heads after each iteration can be obtained using the cumulative sum (or cumsum).
- The cumulative sum at position a , where $a \geq 1$, of an array X is:

$$\sum_{i=0}^a X[i]$$

- The `cumsum()` function is available through the package `numpy`.

```

random_trials = [random.choice([0, 1]) for _ in range(1000)]
cum_sum_random_trials = np.cumsum(random_trials)

print("-" * 30)
print(random_trials[0:10])

print("-" * 30)
print(cum_sum_random_trials[0:10].tolist())

```

```

-----
H T
[1, 1, 0, 0, 0, 0, 0, 1, 1, 1] ← coin toss results
-----
[1, 2, 2, 2, 2, 2, 2, 3, 4, 5] ← cum. sum
                                = # of Heads

```

```

# Let's normalize `cum_sum_random_trials` by the number of trials in each position
cumulative_prop_heads = cum_sum_random_trials / range(1, len(cum_sum_random_trials)+1)

print("=====")
print(random_trials[0:6])
print("-----")
print(cum_sum_random_trials[0:7].tolist())
print("-----")
print(cumulative_prop_heads[0:7].tolist())
print("=====")

=====
[1, 1, 0, 0, 0, 0]
-----
[1, 2, 2, 2, 2, 2]
-----
[1.0, 1.0, 0.6666666666666666, 0.5, 0.4, 0.3333333333333333, 0.2857142857142857]
=====

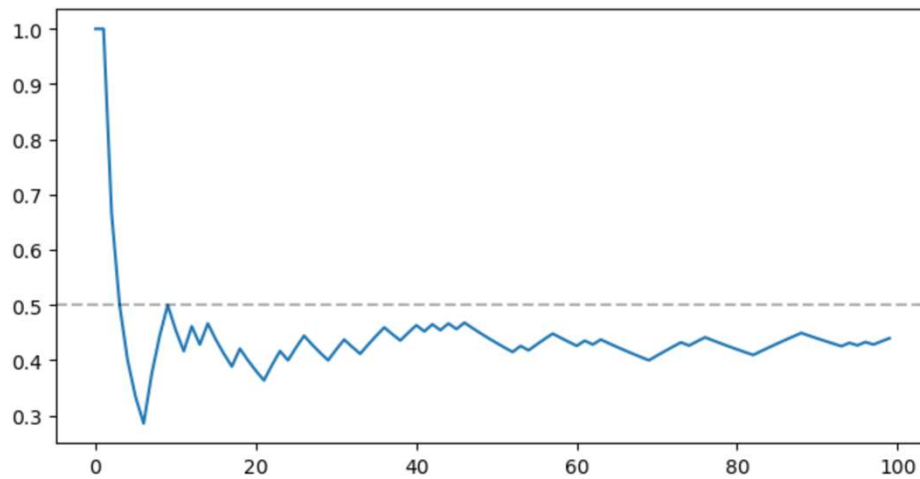
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8, 4))
```

```
plt.plot(range(100), cumulative_prop_heads[0:100])
```

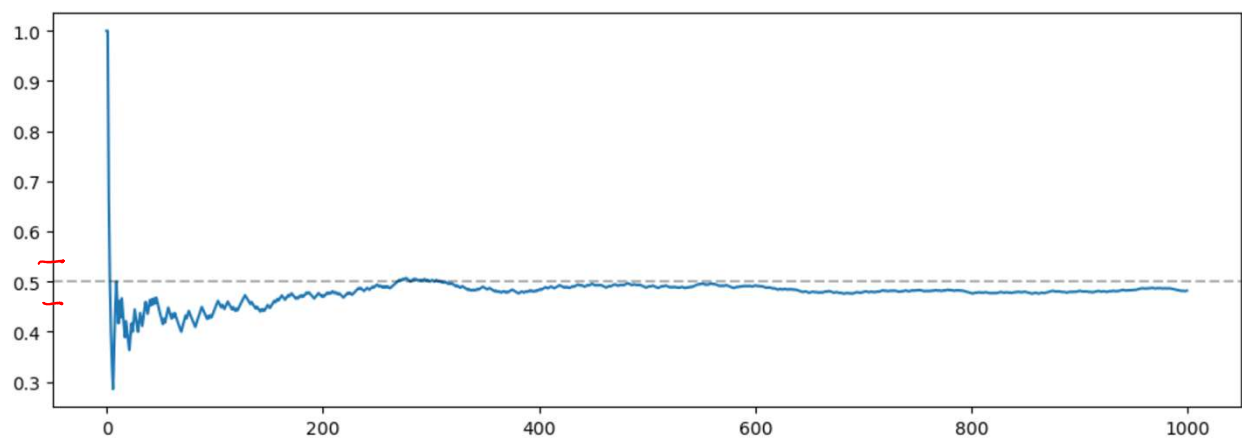
```
plt.axhline(0.5, color='k', linestyle='--', alpha=0.3);
```



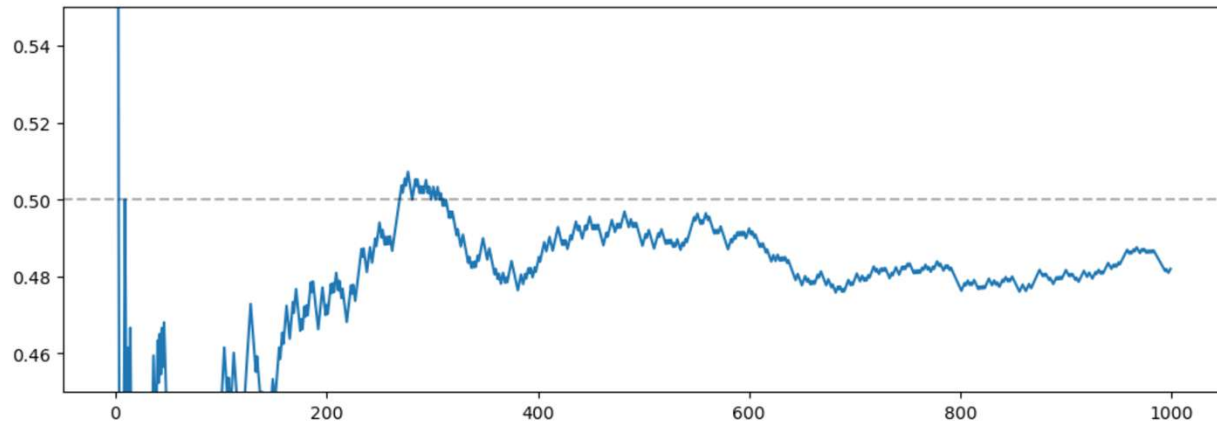
```
plt.figure(figsize=(12, 4))
```

```
plt.plot(range(len(cumulative_prop_heads)), cumulative_prop_heads)
```

```
plt.axhline(0.5, color='k', linestyle='--', alpha=0.3);
```



```
plt.figure(figsize=(12, 4))
plt.plot(range(len(cumulative_prop_heads)), cumulative_prop_heads)
plt.axhline(0.5, color='k', linestyle='--', alpha=0.3);
plt.ylim(0.45, 0.55);
```



Long-run frequency

- Even after 1000 draws, the proportion of heads is not exactly 0.5.
 - A run of 1000 flips is still just a random sample, and there is no guarantee that the relative frequency in the random sample will match the underlying probability of the event.
- However, in the “much longer run,” the frequency will tend to approximate the theoretical probabilities, $p(H)$ and $p(T)$.

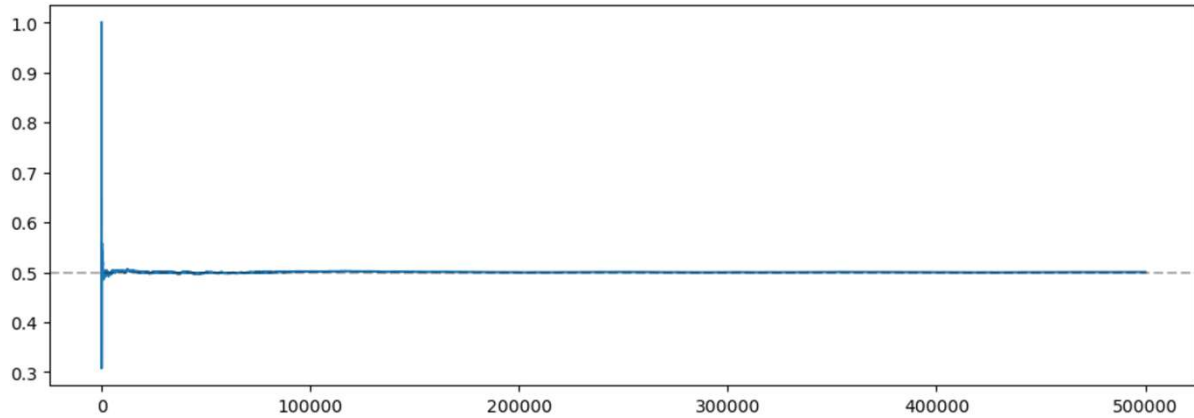
```

random_trials = [random.choice([0, 1]) for _ in range(500_000)]

cum_sum_random_trials = np.cumsum(random_trials)
cumulative_prop_heads = cum_sum_random_trials / range(1, len(cum_sum_random_trials)+1)

plt.figure(figsize=(12, 4))
plt.plot(range(len(cumulative_prop_heads)), cumulative_prop_heads)
plt.axhline(0.5, color='k', linestyle='--', alpha=0.3);

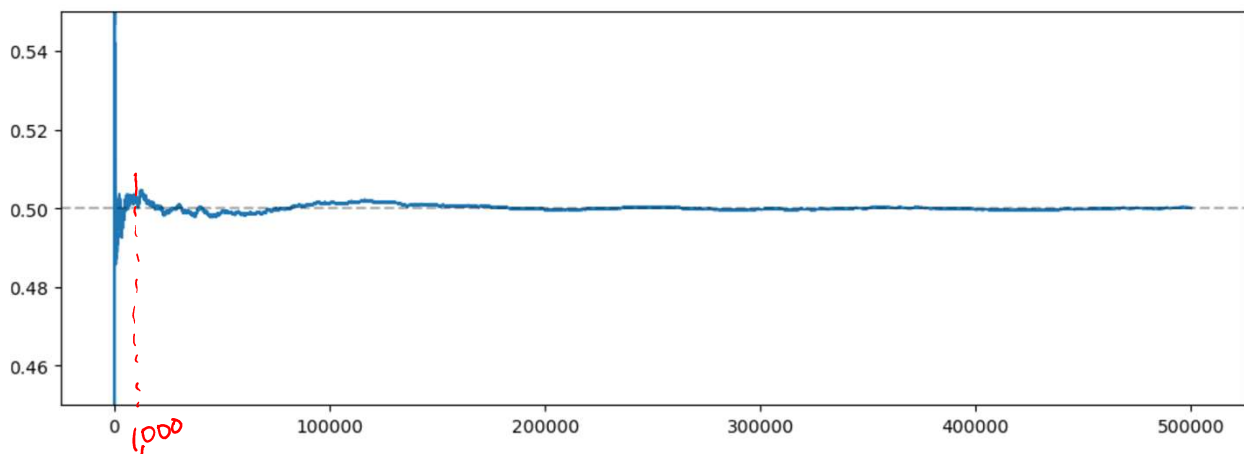
```



```

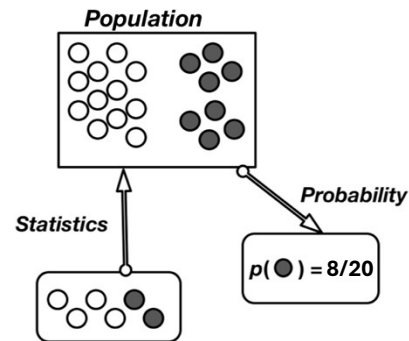
plt.figure(figsize=(12, 4))
plt.plot(range(len(cumulative_prop_heads)), cumulative_prop_heads)
plt.axhline(0.5, color='k', linestyle='--', alpha=0.3);
plt.ylim(0.45, 0.55);

```



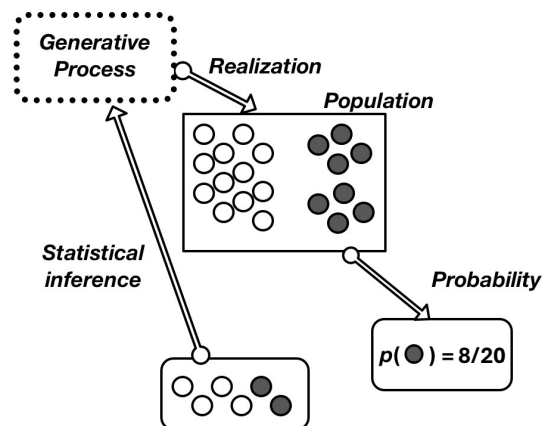
Probability versus Statistics

- Probability is a theoretical branch of mathematics, while statistics is applied mathematics trying to make sense of real-world observations.
- The following is useful although perhaps simplistic distinction between probability and statistics.
 - Probability deals with predicting the likelihood of future events, while statistics analyzes the frequency of past events.



The underlying generative process

- Populations are dynamic. How are new members generated?
- The generative process summarizes the population as it describes how new members can be “realized.”



Using simulation to infer a statistic for generative process

- It's often challenging to mathematically derive event(s) probability(ies).
 - May involve conditional and hierarchical dependencies.
 - Flip a coin, select a die accordingly, then select a distribution, etc.
- Often, it is much simpler to simulate the problem a large number of times to derive the desired probabilities.
 - Challenge with simulation: We need to know that we converged toward the correct solution.
 - We cannot flip a coin 10 times and accept the resulting probability.
 - How do we know we converged onto the correct solution?

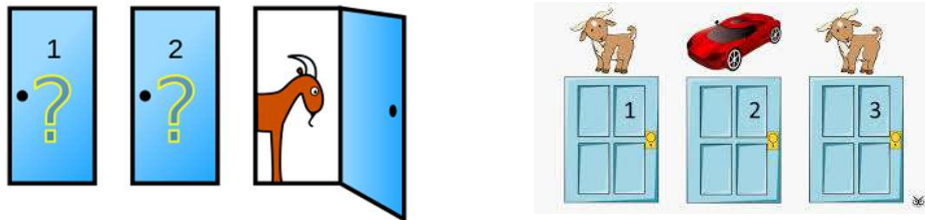
Using simulation to infer a statistic for generative process: Example

- Solving the Monty Hall Problem
 - A problem based on television game show "Let 's Make a Deal"
 - Named after the show's original host, Monty Hall

The Monty Hall problem

“Suppose you are a contestant on a game show and are presented with three doors. Behind one door is a car (the prize) and behind the other two doors are goats. You select one door — say Door 1 — but do not open it. The host, who knows what is behind each door, then opens one of the two remaining doors, say Door 3, revealing a goat. The host then offers you a choice: remain with your original selection or switch to the other unopened door.”

Is it to your advantage to switch your initial choice?



Simulate the problem

- Question: Is it to your advantage to switch your choice?
 - We can easily simulate the problem and answer the question using the following approach:
 1. Randomly place the car behind one of the doors.
 2. Pick a door at random.
 3. Reveal a door with a goat.
 4. Apply a strategy: either
 - 4.1 Switch to the other unopened door, or
 - 4.2 Stick with the initial choice.
- Repeat the process few thousands times and evaluate whether 4.1 is a better strategy than 4.2.

```
def monty_hall(n=10000, switch=False):
    nb_wins = 0
    doors = np.array(["GOAT", "CAR", "GOAT"])
    for _ in range(n):
        # Randomly place the car behind the doors by shuffling
        random.shuffle(doors)

        # Contestant randomly picks a car
        my_pick = random.choice([0, 1, 2])

        # Host opens a door with a goat (not the chosen door, not the car)
        remaining_doors = [i for i in range(3) if i != my_pick and doors[i] == "GOAT"]
        reveal_door = random.choice(remaining_doors)

        # Switch or stick
        if switch:
            my_pick = list(set([0, 1, 2]).difference([my_pick, reveal_door]))[0]

        # Check if the contestant wins
        if doors[my_pick] == "CAR":
            nb_wins += 1
    return nb_wins
```

monty_hall(n=100_000, switch=False)
33248

monty_hall(n=100_000, switch=True)
66659

Which strategy is better?

- The previous does not directly tell us which strategy is better.
 - It simply shows that out of 100,000 experiments, we won nw_1 times by switching and nw_2 times by keeping the initial selection.
 - It shows that in the simulation, $nw_1 > nw_2$.
- Is this always the case?
 - How often will we expect the “Switch” strategy to be better?
- What is the probability of $nw_1 > nw_2$?
 - I.e., in the long run, how often should we expect $nw_1 > nw_2$?

```

nb_trials = 500
nb_wins_switch = []
for _ in range(nb_trials):
    nb_wins_switch.append(monty_hall(n=1000, switch=True))

nb_wins_switch[0:10]

```

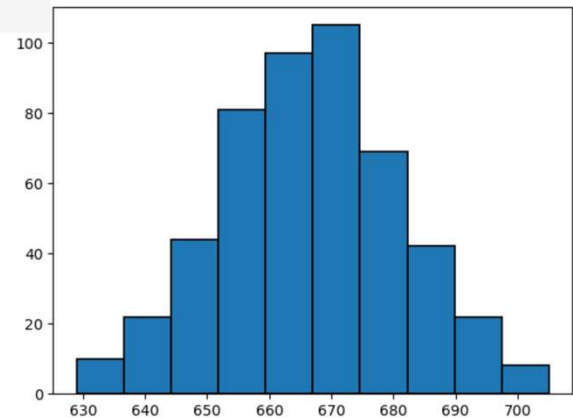
```
[663, 663, 673, 700, 656, 674, 662, 661, 663, 660]
```

```

plt.hist(nb_wins_switch, edgecolor='black', linewidth=1.2)
plt.show()

```

500 values representing the number of wins out of 1,000 games when always switching the initial choice.



```

def compare_strategy(batches=1_000, trials_per_batch=10):
    """
    Estimate probability that switching is better than sticking.
    """
    nb_better_occure = 0
    for i in range(batches):
        nb_wins_switch = monty_hall(trials_per_batch, switch=True)
        nb_wins_stick = monty_hall(trials_per_batch, switch=False)
        if nb_wins_switch >= nb_wins_stick:
            nb_better_occure += 1
    return(nb_better_occure / batches)

```

```
compare_strategy(trials_per_batch=1)
```

```
0.889
```

```
compare_strategy(trials_per_batch=1)
```

```
0.894
```

```
nb_trials = 500
prob_wins_switch = []
for _ in range(nb_trials):
    prob_wins_switch.append(compare_strategy(trials_per_batch=1))
prob_wins_switch[0:10]

[0.891, 0.886, 0.906, 0.886, 0.877, 0.874, 0.882, 0.89, 0.886, 0.893]
```

*500 trials of comparing the 2 strategies
1,000 times*

of game play in each comparison

↓ = 10

```
nb_trials = 500
prob_wins_switch = []
for _ in range(nb_trials):
    prob_wins_switch.append(compare_strategy())
prob_wins_switch[0:10]

[0.969, 0.969, 0.959, 0.966, 0.961, 0.96, 0.964, 0.955, 0.963, 0.969]
```